

Implementation Specification for Automated Unitree L1 Experimental Evaluation

Robot Operating System 2 Humble and Docker on Ubuntu

Operator procedure, software architecture, analysis requirements, and Codex development brief

Item	Definition
Prepared for	Pascual Asio Serrano - Research internship, Institute of Systems and Robotics, University of Coimbra
Laboratory context	Human-Centered Mobile Robotics Laboratory
Target environment	Ubuntu host; Docker container based on Ubuntu 22.04 Jammy; Robot Operating System 2 (ROS 2) Humble
Sensor	Unitree L1 three-dimensional light detection and ranging sensor with inertial measurement unit
Document version	1.0 - 6 August 2026
Primary objective	Create a reproducible software system that acquires, validates, analyses, and reports the remaining experiments identified by the evidence-based assessment.

Scientific scope

The software must measure sensor and pipeline behaviour. It must not manufacture a conclusion. Raw observations, calculated quantities, engineering acceptance criteria, and scientific interpretation remain separate in every output.

Required experimental modules

- Planar-target range bias, repeatability, point-to-plane dispersion, and valid-return behaviour.
- Time-dependent angular coverage of the non-repetitive scan.
- Long-duration ROS 2 stream stability and restart reliability.
- Trajectory and map evaluation against independent ground truth when available; otherwise relative repeatability only.

1. Purpose and scientific boundary

Published evidence supports the Unitree L1 for indoor navigation-scale perception under favourable motion and processing conditions. However, four quantities remain insufficiently verified: range accuracy across controlled conditions, coverage convergence, long-duration ROS 2 stability, and trajectory accuracy against an independent reference. The present document converts these gaps into an executable software and laboratory workflow.

The implementation uses ROS 2 topics for continuous sensor streams, rosbag2 for immutable recording, and offline analysis for reproducibility. ROS 2 provides configurable Quality of Service (QoS) policies and topic statistics, but the evaluation package must also calculate its own per-message metrics because built-in statistics do not cover point validity, effective point rate, plane residuals, or experimental metadata [2]-[4].

The Unitree L1 software publishes point-cloud and inertial data. The official software documentation gives default names equivalent to `/unilidar/cloud` and `/unilidar/imu`, with frames `unilidar_lidar` and `unilidar_imu`. These values must remain configurable because the working Coimbra deployment may use remapped topics [1].

1.1 Non-negotiable scientific rules

- The independent experimental unit is one complete acquisition, not one point and not one message. Per-point sample counts must never be used to inflate statistical significance.
- Raw rosbags are immutable. Re-running an analysis creates new derived outputs; it never rewrites source data.
- A manufacturer specification is a comparison target, not a measured result.
- A trajectory generated from the same L1 point cloud and inertial data is not independent ground truth.
- The software must preserve failed and excluded runs with an explicit status and reason.
- Every acronym is expanded at first occurrence in user-facing reports.
- Automatic conclusions are limited to rule-based evidence statements. A human researcher validates the final interpretation.

1.2 Division of responsibilities

Responsibility	Operator: Pascual	Codex implementation
Physical geometry	Mount the sensor and target; measure reference distance, angle, target dimensions, and origin offset.	Validate that required metadata are present; never infer unmeasured geometry.
Experimental conditions	Control motion, lighting, material, warm-up, and repetitions.	Prompt for each condition, enforce the matrix, and prevent accidental overwriting.
Sensor communication	Verify cabling, network interface, and the working Unitree driver.	Discover topics, types, frames, QoS compatibility, and message fields.
Data acquisition	Confirm the scene is ready and approve start/stop.	Launch monitoring and rosbag2 recording; log timing, resources, and anomalies.
Analysis	Review the region of interest and exclusions.	Perform deterministic calculations, uncertainty propagation, figures, and machine-readable summaries.
Interpretation	Approve the scientific conclusion with supervisors.	Generate cautious candidate statements linked to metrics and stated limitations.

2. Required software architecture

Codex should implement one version-controlled repository. The Unitree driver remains an external dependency or a mounted source workspace. The evaluation code wraps the driver; it does not alter the driver during the measurement campaign.

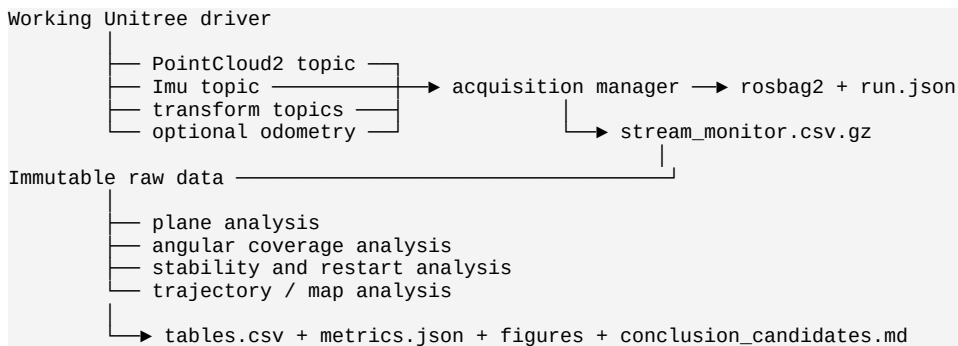
2.1 Repository structure

```

unitree_l1_evaluation/
├── README.md
├── LICENSE
├── docker/
│   ├── Dockerfile
│   ├── compose.yaml
│   └── cyclonedds.xml          # only if the current setup uses Cyclone DDS
├── config/
│   ├── default.yaml
│   ├── experiment_matrix.yaml
│   └── schemas/
│       ├── manifest.schema.json
│       └── run.schema.json
├── src/l1_evaluation/
│   ├── package.xml
│   ├── setup.py
│   ├── resource/l1_evaluation
│   ├── launch/evaluation.launch.py
│   └── l1_evaluation/
│       ├── cli.py
│       ├── configuration.py
│       ├── preflight.py
│       ├── run_manager.py
│       ├── stream_monitor.py
│       ├── bag_reader.py
│       ├── pointcloud.py
│       ├── plane_analysis.py
│       ├── coverage_analysis.py
│       ├── stability_analysis.py
│       ├── trajectory_analysis.py
│       ├── uncertainty.py
│       ├── reporting.py
│       └── provenance.py
├── tests/
│   ├── test_plane_analysis.py
│   ├── test_coverage_analysis.py
│   ├── test_stability_analysis.py
│   ├── test_trajectory_analysis.py
│   ├── test_schemas.py
│   └── test_rosbag_smoke.py
├── scripts/
│   ├── build.sh
│   ├── preflight.sh
│   ├── run_test.sh
│   ├── analyse.sh
│   └── make_report.sh
└── data/
    ├── raw/                  # immutable rosbags
    ├── metadata/
    ├── derived/
    ├── results/
    └── logs/

```

2.2 Processing flow



The acquisition and analysis paths are intentionally separate. This separation permits re-analysis without repeating the experiment and prevents an algorithm change from modifying the raw evidence.

2.3 Command-line interface

```
l1-eval preflight --config config/default.yaml
l1-eval plan --matrix config/experiment_matrix.yaml
l1-eval run planar --condition A1_D2_WHITE_0 --repeat 1
l1-eval run coverage --repeat 1
l1-eval run stability --duration 3600 --restart-cycles 5
l1-eval run trajectory --route LAB_LOOP --repeat 1
l1-eval analyse planar --manifest data/metadata/manifest.csv
l1-eval analyse coverage --manifest data/metadata/manifest.csv
l1-eval analyse stability --manifest data/metadata/manifest.csv
l1-eval analyse trajectory --manifest data/metadata/manifest.csv
l1-eval report --all --output data/results/report
```

Each command returns a non-zero exit status when a mandatory check fails. Human-readable output is concise; detailed errors are written to a timestamped log.

3. Docker and ROS 2 Humble environment

The recommended base image is `ros:humble-ros-base-jammy`. The official image is based on Ubuntu 22.04 Jammy and includes the basic ROS development tools. The image tag and digest must be recorded in each campaign manifest [5].

3.1 Dockerfile requirements for Codex

- Start from `ros:humble-ros-base-jammy` and pin the image digest in the final campaign version.
- Install `colcon`, `rosdep`, `roslaunch`, `sensor message Python utilities`, `diagnostic messages`, `transform libraries`, and the chosen ROS 2 middleware package.
- Install only analysis dependencies required by the implementation: `NumPy`, `SciPy`, `pandas`, `Matplotlib`, `PyYAML`, `jsonschema`, `psutil`, and `pytest`.
- Create a non-root user that matches the host user identifier when practical. Keep the ability to access the network interface used by the L1.
- Copy dependency files before source code to preserve Docker build cache.
- Write the exact Debian package list and Python package list to `/opt/build-metadata` at build time.

```
FROM ros:humble-ros-base-jammy

SHELL ["/bin/bash", "-o", "pipefail", "-c"]
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3-colcon-common-extensions python3-rosdep python3-pip git \
    ros-humble-roslaunch ros-humble-roslaunch-storage-default-plugins \
    ros-humble-sensor-msgs-py ros-humble-tf2-ros \
    ros-humble-diagnostic-msgs ros-humble-rmw-cyclonedds-cpp \
    && rm -rf /var/lib/apt/lists/*

RUN python3 -m pip install --no-cache-dir \
    numpy scipy pandas matplotlib pyyaml jsonschema psutil pytest

WORKDIR /workspace
COPY src/ /workspace/src/
RUN source /opt/ros/humble/setup.bash && colcon build --symlink-install

ENTRYPOINT ["/ros_entrypoint.sh"]
CMD ["bash"]
```

The exact package set may be adjusted after a build test. Codex must not silently upgrade ROS 2, the Unitree driver, or the middleware once the campaign configuration is frozen.

3.2 Docker Compose requirements

```
services:
  evaluation:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    network_mode: host
    environment:
      ROS_DOMAIN_ID: ${ROS_DOMAIN_ID:-17}
      RMW_IMPLEMENTATION: ${RMW_IMPLEMENTATION:-rmw_cyclonedds_cpp}
      L1_NETWORK_INTERFACE: ${L1_NETWORK_INTERFACE:-enp3s0}
    volumes:
      - ../config:/workspace/config:ro
      - ../data:/workspace/data
      - ../src:/workspace/src:ro
      - ${UNITREE_DRIVER_WS:-../unitree_driver_ws}:/workspace/unitree_driver_ws:ro
    stdin_open: true
    tty: true
    stop_grace_period: 20s
```

Host networking is appropriate on a Linux host when the sensor communicates over Ethernet. It avoids multicast and Data Distribution Service discovery problems caused by network-address translation. Privileged mode must not be enabled by default. Add a specific device mapping or network capability only when the working driver demonstrably requires it.

3.3 Configuration freeze

- Record host operating system, kernel, Docker version, image name and digest, ROS 2 distribution, middleware implementation, Unitree driver commit, evaluation commit, and all YAML configuration hashes.
- Record the network interface and Internet Protocol addresses used by the sensor and host.
- Keep the current working middleware. A campaign is not the moment to compare middleware implementations unless that comparison is a separate experiment.
- Synchronize clocks before tests that involve an external reference. Record the synchronization method and residual offset.

4. Configuration and data model

4.1 Required YAML configuration

```

experiment:
  campaign_id: 20260806_COIMBRA_L1
  operator: Pascual Asio Serrano
  sensor_model: Unitree L1
  sensor_variant: UNKNOWN          # must be replaced by PM or RM after inspection
  serial_number: REQUIRED
  warmup_s: 600

ros:
  cloud_topic: /unilidar/cloud
  imu_topic: /unilidar/imu
  tf_topics: [/tf, /tf_static]
  wheel_odom_topic: null
  lidar_odom_topic: null
  ground_truth_topic: null
  ros_domain_id: 17
  expected_cloud_type: sensor_msgs/msg/PointCloud2
  expected_imu_type: sensor_msgs/msg/Imu

recording:
  storage_id: sqlite3
  compression: none
  output_root: /workspace/data/raw
  monitor_sample_period_s: 1.0
  graceful_stop_timeout_s: 15

point_filter:
  min_range_m: 0.10
  max_range_m: 20.0
  required_fields: [x, y, z]
  optional_intensity_fields: [intensity, reflectivity, grayscale]

plane:
  roi_mode: angular
  azimuth_center_deg: 0.0
  elevation_center_deg: 0.0
  azimuth_half_width_deg: 8.0
  elevation_half_width_deg: 8.0
  ransac_threshold_m: 0.015
  ransac_iterations: 500
  minimum_inlier_ratio: 0.60
  edge_margin_fraction: 0.15

coverage:
  grid_deg: 2.0
  sensitivity_grid_deg: 1.0
  windows_s: [0.1, 0.25, 0.5, 1, 2, 5, 10, 30]
  final_reference_window_s: 30

stability:
  duration_s: 3600
  restart_cycles: 5
  gap_warning_s: 0.5
  gap_failure_s: 1.0
  resource_sample_period_s: 5.0

trajectory:
  alignment: se3_no_scale
  time_offset_mode: fixed
  fixed_time_offset_s: 0.0
  rpe_intervals_s: [1.0, 5.0]
  allow_absolute_claims_without_ground_truth: false

```

4.2 Manifest and run record

The manifest contains one row per acquisition. A separate run.json stores nested configuration and provenance. The following fields are mandatory when relevant:

Category	Mandatory fields
Identity	campaign_id, condition_id, run_id, operator, date_time_utc, sensor variant, serial number

Category	Mandatory fields
Physical condition	reference distance, nominal incidence, target material, target size, sensor orientation, environment
Software	host information, Docker digest, ROS 2 distribution, middleware, driver commit, analysis commit, configuration hashes
Recording	bag path, topics, duration, start and stop status, message counts, bag reopened status
Quality	valid, excluded, exclusion reason, anomaly notes, operator confirmation
Provenance	SHA-256 hash of every raw bag directory and configuration file

Secure Hash Algorithm 256-bit (SHA-256) values are calculated after a bag closes successfully. The software refuses to overwrite an existing run identifier.

5. Preflight and acquisition manager

5.1 What Pascual must do before Codex software starts

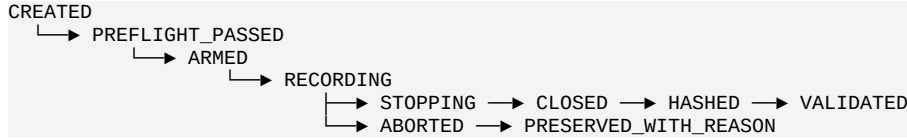
1. Photograph the sensor label, mounting, axes, target, and reference-measurement geometry.
2. Confirm the L1 Precision Measurement or Remote Measurement variant and enter it in configuration.
3. Clean the optical window and warm the sensor for at least ten minutes.
4. Verify that the Unitree driver already publishes usable point-cloud and inertial messages.
5. Measure the offset between the physical point used by the distance instrument and the LiDAR frame origin.
6. Create the first condition in the experiment matrix and confirm the output disk has sufficient space.

5.2 What Codex must implement in preflight

- Discover configured topics and verify their message types.
- Inspect publisher QoS endpoints and create compatible subscriptions. SensorDataQoS or an equivalent best-effort profile is a likely default, but the actual publisher policy is authoritative [3].
- Read at least 100 point-cloud messages and 1,000 inertial messages.
- Verify that x, y, and z fields exist; identify optional intensity-like fields without assuming their name.
- Check finite point counts, header frame identifiers, timestamp monotonicity, reception intervals, and approximate data rate.
- Verify transform availability for requested frames. The static experiments may remain entirely in the LiDAR frame.
- Record a ten-second pilot bag, reopen it with rosbag2, count messages, and decode one message of each required type.
- Print a concise PASS, WARNING, or FAIL report. Only PASS permits a formal run.

Preflight check	FAIL condition	WARNING condition
Topic presence	Point-cloud or inertial topic absent.	Optional transform or odometry topic absent.
Message type	Configured type does not match published type.	Additional undocumented fields are present.
Timestamps	Backward or non-finite header timestamps.	Header time and reception time use visibly different clocks.
Point data	x, y, or z cannot be decoded.	High invalid fraction in the pilot scene.
Rosbag2	Pilot cannot be reopened or required messages are missing.	Storage plugin differs from planned plugin.
Metadata	Sensor serial number or campaign identifier missing.	Optional room temperature unavailable.

5.3 Acquisition state machine



Signals must be handled explicitly. When the operator presses Control-C, the manager first stops rosbag2 gracefully, waits for metadata to flush, stops the monitor, validates the bag, writes the run record, and only then exits.

6. Module A - planar-target metrology

6.1 Operator procedure

7. Use a rigid matte planar target at least 0.8 m by 0.8 m. Mark a central region that will remain clear of edges.
8. For the distance series, place the target at 0.5, 1, 2, 3, and 5 m. Add 8 or 10 m only when geometry and reference measurement remain reliable.
9. At a fixed distance of 2 or 3 m, test nominal incidence angles of 0, 30, and 60 degrees.
10. At 2 m and near-normal incidence, test at least a matte white and a matte black target.
11. Acquire five independent repetitions per condition. Reposition or fully stop and restart recording between repetitions.
12. Measure reference distance twice. Repeat the placement when the two values differ by more than 3 mm.
13. Enter reference-instrument resolution, calibration information if available, origin-offset uncertainty, positioning uncertainty, and alignment uncertainty.

6.2 Codex implementation requirements

- Record 20 seconds of raw point cloud and inertial data per repetition.
- Extract the configured angular or Cartesian region of interest. Save a diagnostic image or point-cloud file for operator review.
- Remove non-finite points and apply only the configured physical range limits.
- Fit a plane by Random Sample Consensus (RANSAC), then refit the inliers by singular value decomposition.
- Calculate the actual incidence angle from the fitted normal and the line from the origin to the plane centroid.
- Calculate one distance estimate per message and one robust acquisition estimate. The acquisition estimate is the statistical unit.
- Produce residual histograms and a three-dimensional diagnostic plot only for failed or selected representative runs; do not create unnecessary figures.

6.3 Required calculations

For a normalized plane n transpose p plus d equals zero, the estimated target distance is the absolute value of d . The signed error for acquisition i is:

$$e_i = d_{\text{hat}_i} - d_{\text{ref}_i}$$

Report the mean bias across independent acquisitions, its standard deviation, the median, the minimum and maximum, and a 95% confidence interval. Within-run dispersion is reported separately as the point-to-plane root-mean-square error and median absolute deviation.

$$\begin{aligned}
 u_{\text{ref}} &= \sqrt{u_{\text{meter}}^2 + u_{\text{origin}}^2 + u_{\text{position}}^2 + u_{\text{alignment}}^2} \\
 U_{\text{ref},95} &\approx 2 \times u_{\text{ref}} \\
 E_{\text{conservative}} &= |\text{mean_bias}| + U_{\text{ref},95}
 \end{aligned}$$

A tested condition is described as compatible with the manufacturer-stated ± 2 cm value only when the confidence interval for bias and the reference uncertainty both remain within the ± 2 cm interval. A condition is incompatible when the full interval lies outside. All overlap cases are inconclusive. The software must use these words exactly and must state that the conclusion applies only to the tested condition.

Output	Definition	Scientific meaning
bias_m	Mean signed acquisition error.	Systematic displacement under one condition.
repeatability_m	Standard deviation across acquisition estimates.	Reproducibility of independent recordings.
plane_rmse_m	Root-mean-square point-to-plane residual.	Local geometric dispersion; not range bias.

Output	Definition	Scientific meaning
plane_mad_m	Median absolute point-to-plane residual.	Robust local dispersion.
valid_fraction	Finite inlier-capable points divided by received points in the region.	Return robustness in the selected target region.
actual_incidence_deg	Angle derived from fitted plane geometry.	Observed incidence, distinct from nominal placement.

7. Module B - non-repetitive angular coverage

7.1 Operator procedure

14. Place the sensor motionless in a structured room with visible walls and ceiling.
15. Prevent people or moving objects from entering the scene.
16. Record three independent 60-second runs without changing the sensor orientation.
17. Optionally repeat with a 30-degree tilt as a separate condition.

7.2 Codex implementation requirements

- Convert every valid point to azimuth and elevation.
- Accumulate angular-cell occupancy over 0.1, 0.25, 0.5, 1, 2, 5, 10, and 30 seconds.
- Use a 2-degree by 2-degree grid for the primary result and repeat the calculation at 1 degree by 1 degree as a sensitivity analysis.
- Define observable cells as cells occupied by the pooled final 30-second reference for that acquisition. Do not divide by the full theoretical field of view because room geometry and target visibility limit observable returns.
- Calculate coverage fraction, hit-count coefficient of variation, normalized occupancy entropy, valid point rate, and t50, t90, and t95 convergence times.
- Aggregate the three acquisitions by reporting median and range or mean and standard deviation. Preserve each individual curve.

```
coverage(t) = occupied_cells(t) / occupied_cells(30 s)
CV_hits(t) = standard_deviation(hit_count) / mean(hit_count)
H_norm(t) = -sum(p_k log(p_k)) / log(K)
```

The analysis answers how quickly the observed indoor scene is sampled. It does not reconstruct the theoretical emitted-ray pattern. The generated conclusion must therefore use the term observed angular coverage.

8. Module C - ROS 2 stream stability and restart reliability

8.1 Operator procedure

18. Use the final frozen driver and Docker image.
19. Keep the sensor static in a representative indoor scene.
20. Record one uninterrupted 60-minute sequence.
21. After the long run, perform five complete driver restart cycles without power-cycling unless a power-cycle test is explicitly planned.
22. Record every manual intervention, network reconnection, or software restart.

8.2 Codex stream monitor

The monitor subscribes to point-cloud and inertial topics with compatible QoS. It writes one compressed row per message or one-second aggregate, together with process and container resource samples. The monitor must use monotonic host time for reception intervals and message header time for source timing. These clocks are never mixed without an explicit offset model.

Metric	Implementation
Reception period	Difference between consecutive monotonic reception times; report mean, median, standard deviation, 95th percentile, maximum, and gaps.
Header period	Difference between consecutive message header timestamps; count backward, duplicate, and discontinuous values.

Metric	Implementation
Message age	Reception wall-clock time minus header time only when clock compatibility is verified; otherwise mark unavailable.
Useful point rate	Sum of finite, in-range points divided by elapsed monotonic time.
Invalid fraction	Non-finite or physically invalid points divided by decoded points.
Serialized throughput	Serialized message bytes divided by elapsed time.
Availability	Observed time not contained in configured gap intervals divided by total test time. This is an estimated stream availability, not network packet loss.
Resources	Process-tree and container central processing unit use, resident memory, and available diagnostic temperature.
Restart latency	Time from driver launch to first valid point cloud and first valid inertial message.

8.3 Stability analysis and decision rules

- Divide the 60-minute run into twelve five-minute bins. Report each metric by bin and for the full test.
- Estimate robust temporal drift with a Theil-Sen slope for message rate, invalid fraction, processor use, and memory.
- Flag any backward timestamp, driver crash, decoder exception, or gap above one second as a major anomaly.
- Compare inertial rate with the manufacturer-stated 250 Hz output, but report both observed rate and uncertainty rather than forcing a pass or fail.
- Compare useful point rate with the stated effective rate only under a scene with sufficient returns. Low valid return rate in an empty or low-reflectance scene is not automatically a software failure.
- Report restart success as x out of five and give median, minimum, and maximum startup latency.

An engineering stability label may be generated using pre-registered thresholds: STABLE when no major anomaly occurs, estimated availability is at least 99.9%, every restart succeeds, and no sustained memory increase is detected; DEGRADED when the stream remains usable but one criterion fails; FAILED when recording is interrupted, timestamps move backward, or repeated restarts fail. The report must identify these as project acceptance criteria, not universal Unitree specifications.

9. Module D - trajectory and map evaluation

9.1 Evidence hierarchy

Level	Reference	Permitted claim
1	Motion capture, total station, or surveyed reference trajectory.	Absolute trajectory error within the documented reference uncertainty.
2	Independently calibrated reference sensor and validated odometry or inertial navigation system.	Comparative trajectory accuracy relative to the reference system.
3	Wheel odometry and surveyed landmarks.	Partial external validation with clearly stated limitations.
4	Repeated L1-only traversals.	Repeatability and relative geometric consistency only.

Point-LIO or another estimator that uses the same L1 point cloud and inertial measurement unit may serve as an operational baseline, but not as independent ground truth. The software must encode the evidence level in every trajectory result.

9.2 Operator procedure

23. Select a 20 to 40 m indoor loop with a corridor, a 90-degree turn, doors, and a return to the start.
24. Mark the start and fixed waypoints. Keep speed near 0.2 to 0.3 m/s and repeat the route at least three times.
25. Rigidly mount and calibrate any external reference sensor. Record the extrinsic transformation and its uncertainty.
26. Synchronize clocks or estimate and validate a constant time offset before calculating trajectory errors.

27. Record point cloud, inertial data, transforms, wheel odometry, estimator output, and ground truth simultaneously.

9.3 Codex trajectory analysis

- Support nav_msgs/Odometry and geometry_msgs/PoseStamped input through configurable adapters.
- Interpolate positions to common timestamps and use spherical linear interpolation for orientation.
- Estimate or apply the documented time offset. Save a plot of residual alignment versus tested offset when automatic estimation is used.
- Align trajectories with a rigid Special Euclidean group transformation in three dimensions without scale. Permit a planar Special Euclidean group transformation in two dimensions only when the experiment is explicitly planar.
- Calculate absolute trajectory error, relative pose error at 1 and 5 seconds, final loop-closure displacement, final orientation error, coverage of matched timestamps, and data-exclusion fraction.
- For level 4 evidence, suppress absolute-error headings and report only pairwise repeatability and loop closure.

```
ATE_RMSE = sqrt(mean(||translation(T_ref_i^-1 T_est_i)||^2))
RPE( $\Delta$ ) = error of relative motion over interval  $\Delta$ 
matched_fraction = matched_reference_samples / requested_samples
```

Map comparison is secondary. If comparable three-dimensional maps are available, Codex may calculate pairwise nearest-neighbour distances after a documented rigid alignment and may report median and 95th percentile distances over preselected static regions. It must not hide manual alignment or compare moving objects, doors, or unobserved regions.

10. Analysis outputs and scientific conclusion logic

10.1 Mandatory outputs

Path	Contents
data/derived/<run_id>/	Decoded per-message metrics, selected region diagnostics, and intermediate arrays.
data/results/tables/	One comma-separated values file per module plus a campaign summary.
data/results/figures/	Print-friendly figures with white backgrounds, black text, and distinguishable line styles.
data/results/metrics.json	Machine-readable values, units, confidence intervals, evidence level, and validity status.
data/results/conclusion_candidates.md	Rule-based statements with supporting metric identifiers and limitations.
data/results/provenance.json	Input hashes, code commit, configuration hash, dependency versions, and execution time.

10.2 Figures that are scientifically necessary

- Range bias versus reference distance with independent-acquisition uncertainty bars.
- Point-to-plane dispersion and valid-return fraction versus incidence and material.
- Observed angular coverage versus accumulation time, plus one angular occupancy map.
- Point-cloud and inertial message periods over the 60-minute run, with anomalies marked.
- Memory and processor use over time only when these data clarify stability.
- Trajectory error or repeatability only when the evidence level and reference are explicit.

Do not generate decorative dashboards, dark backgrounds, three-dimensional plots without a diagnostic purpose, or tables that duplicate a clear figure. Every figure includes units, sample count, condition, repetition definition, and a caption that states the limitation.

10.3 Automatic candidate conclusions

Codex may generate candidate text, but each sentence must be traceable to a rule and metric. Examples follow:

SUPPORTED UNDER TESTED CONDITION:

"At 2 m on the matte white target at 0° nominal incidence, the mean range bias was ... m (95% confidence interval ...), with expanded reference uncertainty ... m. This result is compatible with the manufacturer-stated ± 0.02 m value under this tested condition."

INCONCLUSIVE:

"The uncertainty interval overlaps the ± 0.02 m specification boundary; the present setup cannot establish compatibility or incompatibility."

COVERAGE:

"Observed angular coverage reached 90% of its 30 s reference after ... s (range across three acquisitions: ... to ... s), showing that scan accumulation materially increases scene sampling."

STABILITY:

"During the 60 min test, the stream produced ... major anomalies, estimated availability of ...%, and ... successful restarts out of five. The project stability criterion was therefore"

TRAJECTORY:

"Relative to [reference and evidence level], the aligned trajectory achieved an absolute trajectory error root-mean-square value of ... m. The result does not establish survey-grade accuracy beyond the reference system and tested route."

11. Software verification required from Codex

The analysis code must be tested with synthetic data before it processes the campaign. Passing unit tests is a prerequisite for reporting experimental values.

Test	Synthetic case	Acceptance
Plane fitting	Plane at a known distance and orientation with Gaussian noise and outliers.	Recovered distance and normal agree within specified numerical tolerance.
Uncertainty	Known uncertainty components.	Combined and expanded uncertainty match hand calculations.
Coverage	Predetermined angular cells appearing at known times.	Coverage, t50, t90, t95, coefficient of variation, and entropy are exact.
Stability	Timestamp series with duplicates, backward steps, gaps, and rate drift.	Every anomaly and summary metric is detected correctly.
Trajectory	Known rigid transform, time offset, and added noise.	Alignment recovers the transform and expected error.
Schema	Valid and invalid manifests.	Missing mandatory fields fail with a clear message.
Rosbag smoke	Small generated PointCloud2 and Imu bag.	Reader decodes types and reproduces message counts.

- Run ruff or flake8, mypy where practical, pytest, and colcon test.
- Use deterministic random seeds in tests and record the seed in analysis provenance.
- Fail loudly when units, frames, or message fields are ambiguous.
- Do not catch broad exceptions and continue with partial results without marking the run invalid.
- Document every command needed to rebuild the image and reproduce the report from raw bags.

12. Practical execution plan

Phase	Pascual	Codex deliverable	Completion criterion
1. Scaffold	Provide current driver workspace and topic names.	Repository, Docker image, configuration schema, command-line interface.	Image builds and help command runs.
2. Preflight	Connect sensor and approve a pilot.	Discovery, QoS, decode, pilot bag, PASS/WARNING/FAIL report.	Pilot bag reopens with correct counts.
3. Acquisition	Place target and execute matrix.	Run manager, immutable bag naming, metadata prompts,	No run can exist without manifest and hash.

Phase	Pascual	Codex deliverable	Completion criterion
		graceful stop.	
4. Core analysis	Approve region of interest and exclusions.	Plane, coverage, and stability modules.	Synthetic tests pass and pilot results are plausible.
5. Mobile analysis	Provide route and reference source.	Trajectory adapters, alignment, metrics, evidence-level control.	Known synthetic trajectory is recovered.
6. Reporting	Review scientific wording.	Tables, figures, metrics, provenance, candidate conclusions.	One command reproduces all results.

12.1 Commands Pascual should execute

```
# 1. Build the frozen environment
docker compose -f docker/compose.yaml build

# 2. Start an interactive evaluation shell
docker compose -f docker/compose.yaml run --rm evaluation bash

# 3. Inside the container
source /opt/ros/humble/setup.bash
source /workspace/install/setup.bash
l1-eval preflight --config /workspace/config/default.yaml

# 4. Print the experiment matrix and remaining runs
l1-eval plan --matrix /workspace/config/experiment_matrix.yaml

# 5. Execute one run after placing the physical setup
l1-eval run planar --condition A1_D2_WHITE_0 --repeat 1

# 6. Analyse all completed runs
l1-eval analyse all --manifest /workspace/data/metadata/manifest.csv

# 7. Reproduce the complete report
l1-eval report --all --output /workspace/data/results/report
```

12.2 Checklist before accepting Codex output

- A fresh clone builds from the documented command.
- Topic names, frame identifiers, range limits, and QoS are configuration values, not hidden constants.
- The software never overwrites or deletes raw data.
- All units are present in field names or metadata.
- All four analysis modules have synthetic tests.
- A failed or interrupted run remains visible and is excluded with a reason.
- The report distinguishes observed data, derived metrics, acceptance criteria, and scientific inference.
- The final figures remain readable when printed in grayscale.
- The README explains both the operator procedure and the developer workflow.

13. Copy-paste development brief for Codex

The following instruction can be given to Codex together with this document and the current Unitree driver repository:

Implement the repository specified in "Unitree L1 Experimental Evaluation - Codex Implementation Specification".

Constraints:

1. Target Ubuntu 22.04 Jammy in Docker with ROS 2 Humble.
2. Do not modify the Unitree driver. Treat its workspace as a read-only dependency.
3. Do not invent topic names, frame identifiers, fields, or sensor variants. Read them from YAML and verify them during preflight.
4. Preserve all raw rosbags. Never overwrite or automatically delete experimental data.
5. Implement the command-line interface: preflight, plan, run, analyse, and report.
6. Implement four modules: planar-target metrology, angular coverage, stream stability and restart testing, and trajectory evaluation with evidence-level control.
7. Use one acquisition as the independent statistical unit. Keep per-message and per-point quantities descriptive.
8. Produce CSV/JSON results, print-friendly figures, provenance hashes, and cautious conclusion candidates.
9. Add synthetic unit tests for every numerical method and a small ROS bag smoke test.
10. Fail safely and clearly when units, timestamps, message fields, transforms, or ground-truth status are ambiguous.

Development order:

- A. Inspect the existing repository and current driver launch procedure.
- B. Create the Docker and package scaffold.
- C. Implement configuration schemas and preflight.
- D. Implement acquisition and monitoring.
- E. Implement offline analyses and tests.
- F. Implement reporting and README instructions.

At the end of each stage, run the relevant tests and summarize changed files, commands executed, assumptions, unresolved issues, and the next stage. Do not claim completion until a clean build, all tests, and one end-to-end pilot run succeed.

14. Known limitations and decisions that require supervisors

- The current public evidence does not define a universal pass threshold for ROS 2 message rate, processor use, or map accuracy. Project thresholds must be stated as engineering criteria.
- The official Unitree software gives default topics, but the exact Coimbra driver version and message fields must be measured during preflight.
- A target reflectance value cannot be inferred from colour alone. White and black targets provide practical contrast but do not constitute calibrated reflectance standards.
- Temperature can be analysed only if the sensor or driver publishes it or an external thermometer is recorded.
- Absolute trajectory accuracy requires a reference independent of the evaluated L1 pipeline. Without it, conclusions remain limited to repeatability.
- Publication authorship, data ownership, and release of bags or code require approval from Professor Urbano J. Nunes and Doctor Tiago Barros.

References

- [1] Unitree Robotics, "Unilidar SDK: SDK for Unitree L1 LiDAR," official software repository, accessed 6 August 2026. The repository documents point-cloud and inertial outputs and the configurable ROS 2 topic names.
- [2] Open Robotics, "Recording and playing back data," ROS 2 Humble documentation, accessed 6 August 2026.
- [3] Open Robotics, "Quality of Service settings," ROS 2 Humble documentation, accessed 6 August 2026.
- [4] Open Robotics, "Interfaces: topics, services, actions - topic statistics," ROS 2 Humble documentation, accessed 6 August 2026.
- [5] Open Source Robotics Foundation, "ROS official Docker image," Docker Hub, ros:humble-ros-base-jammy, accessed 6 August 2026.
- [6] Unitree Robotics, Unitree 4D LiDAR-L1 User Manual, version 1.1, June 2024.
- [7] B. Muralikrishnan, "Performance Evaluation of Terrestrial Laser Scanners - A Review," Measurement Science and Technology, vol. 32, no. 7, 072001, 2021, doi: 10.1088/1361-6501/abdae3.
- [8] P. Habibiroudkenar et al., "Non-Repetitive Scan Correction for LiDAR Odometry via Spatial Analysis in CERN Facilities," IEEE Access, vol. 14, pp. 52717-52733, 2026, doi: 10.1109/ACCESS.2026.3680632.
- [9] D. He et al., "Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry," Advanced Intelligent Systems, vol. 5, no. 7, 2200459, 2023, doi: 10.1002/aisy.202200459.

Appendix A - Field run card

Check	Value / confirmation
Campaign and condition identifier	
Sensor variant and serial number	
Optical window clean; warm-up at least 10 min	Yes / No
Mount and frame orientation unchanged	Yes / No
Reference distance and origin offset recorded	
Incidence and material recorded	
Point-cloud and inertial topics present	Yes / No
Preflight status	PASS / WARNING / FAIL
Run identifier does not already exist	Yes / No
No moving person in measurement region	Yes / No
Bag reopened and message counts verified	Yes / No
Run decision	VALID / EXCLUDED / REPEAT
Reason or anomaly	